

C++ Interview Solutions

Concise explanations for the 25 standalone implementations

Pattern	Representative problems
Two pointers / sliding window	Squares, Longest Continuous Subarray, Max Beauty, Most Frequent Element
Hashing / counting	Group Anagrams, Rank Teams, Word Break
Dynamic programming	Word Break, Climbing Stairs, Maximum Subarray
Tree / linked-list traversal	Boundary Tree, LCA III, Palindrome Linked List
Sorting + structured scan	4Sum, Reveal Cards, Max Beauty

1. Sort Integers by The Power Value

Pattern	Memoized recursion + sorting	Best container	unordered_map, vector
Time	$O(m \log m + \text{total unique Collatz states}), \text{ where } m = \text{hi-lo}+1$	Space	$O(m + \text{cached states})$

Core idea: Compute each integer's Collatz power once, cache it, then sort by (power, value).

Why it works: The memo avoids recomputing long Collatz chains for values reached repeatedly.

2. Squares of a Sorted Array

Pattern	Two pointers	Best container	vector
Time	$O(n)$	Space	$O(n)$ output

Core idea: The largest square must come from either end because the input is sorted. Compare $|left|$ and $|right|$ and fill the answer from the back.

Why it works: Negative values can have larger squares than positive middle values, so ordinary left-to-right squaring is not sorted.

3. Longest Continuous Subarray With Absolute Diff $\leq$ Limit

Pattern	Sliding window + monotonic deques	Best container	deque
Time	$O(n)$	Space	$O(n)$ worst case

Core idea: Maintain the maximum and minimum value inside the current window. Shrink the left edge whenever max-min exceeds limit.

Why it works: The decreasing max deque and increasing min deque expose both extremes in $O(1)$.

4. 4Sum

Pattern	Sort + two nested loops + two pointers	Best container	vector
Time	$O(n^3)$	Space	$O(1)$ extra, excluding output/sort stack

Core idea: Fix the first two values, then solve the remaining 2-sum using left/right pointers. Skip duplicates at every level.

Why it works: Sorting lets pointer movement increase or decrease the sum predictably and makes duplicate removal simple.

5. Boundary of Binary Tree

Pattern	Tree traversal	Best container	vector
Time	$O(n)$	Space	$O(h)$ recursion

Core idea: Add root, non-leaf left boundary, all leaves left-to-right, then non-leaf right boundary in reverse.

Why it works: Separating the three boundary pieces prevents duplicates, especially for leaves.

6. Maximum Matrix Sum

Pattern	Greedy parity observation	Best container	Simple counters
Time	$O(mn)$	Space	$O(1)$

Core idea: Sum absolute values. If the number of negatives is odd, one value must effectively remain negative; choose the smallest absolute value.

Why it works: A sign-flip operation changes two cells, so only the parity of the number of negatives matters.

7. The kth Factor of n

Pattern	Linear factor scan	Best container	No special container
Time	$O(n)$	Space	$O(1)$

Core idea: Scan from 1 through n , decrement k whenever a divisor is found, and return when k reaches zero.

Why it works: Factors are encountered in increasing order automatically.

8. Create Hello World Function

Pattern	Direct return	Best container	string
Time	$O(1)$	Space	$O(1)$

Core idea: Return the exact required string.

Why it works: There is no state or algorithmic work involved.

9. Longest Palindromic Substring

Pattern	Expand around center	Best container	string
Time	$O(n^2)$	Space	$O(1)$

Core idea: Treat every position as an odd center and every gap as an even center. Expand while characters match and keep the longest interval.

Why it works: Every palindrome has either one center character or one center gap.

10. Reveal Cards In Increasing Order

Pattern	Simulate positions	Best container	queue, vector
Time	$O(n \log n)$	Space	$O(n)$

Core idea: Sort the cards. Use a queue of indices to simulate where each next-smallest card must be placed.

Why it works: Simulating indices is easier than reversing the reveal process on card values.

11. Find the Celebrity

Pattern	Candidate elimination + verification	Best container	No extra container in LeetCode version
Time	$O(n)$	Space	$O(1)$

Core idea: Walk through people and replace the candidate whenever the current candidate knows someone. Then verify the final candidate.

Why it works: If A knows B, A cannot be the celebrity. One pass leaves only one possible candidate.

12. Nested List Weight Sum

Pattern	Depth-first search	Best container	vector
Time	$O(N)$	Space	$O(d)$ recursion depth

Core idea: Recursively visit each item and add integer * current depth.

Why it works: Recursion naturally carries the depth of each nested level.

13. Maximum Beauty of an Array After Applying Operation

Pattern	Sort + sliding window	Best container	vector
Time	$O(n \log n)$	Space	$O(1)$ extra

Core idea: After sorting, values can all be made equal when $\max - \min \leq 2k$. Maintain the longest window satisfying that condition.

Why it works: Each number may move anywhere within $[x-k, x+k]$, so intervals overlap at a common value exactly under this range condition.

14. Word Break

Pattern	Dynamic programming	Best container	unordered_set, vector
Time	$O(n^2)$ substring checks	Space	$O(n + \text{dictionary})$

Core idea: Let $dp[i]$ mean the prefix $s[0..i)$ can be segmented. For each end position, try earlier valid split points whose suffix is in the dictionary.

Why it works: Each valid prefix can serve as a starting point for the next dictionary word.

15. Valid Parentheses

Pattern	Stack matching	Best container	stack
Time	$O(n)$	Space	$O(n)$

Core idea: Push opening brackets. For every closing bracket, the stack top must contain its matching opener.

Why it works: Nested structures close in reverse order, which is exactly LIFO behavior.

16. Palindrome Number

Pattern	Reverse only half the digits	Best container	Integers only
Time	$O(\log_{10} n)$	Space	$O(1)$

Core idea: Reject negative numbers and nonzero numbers ending in zero. Reverse digits until the reversed half is at least as large as the remaining half.

Why it works: Only half the number must be reversed, avoiding overflow and extra storage.

17. Rank Teams by Votes

Pattern	Frequency ranking + custom sort	Best container	array, 26>
Time	$O(V \cdot T + T \log T \cdot T)$	Space	$O(26^2)$

Core idea: Count how many times each team appears in each rank position. Sort teams lexicographically by those count vectors, then alphabetically on ties.

Why it works: The comparison directly implements the voting rule from first-place votes onward.

18. Frequency of the Most Frequent Element

Pattern	Sort + sliding window	Best container	vector
Time	$O(n \log n)$	Space	$O(1)$ extra

Core idea: Sort values and maintain a window that can all be increased to `nums[right]` using at most k operations.

Why it works: Required operations equal `target*window_size - window_sum`; shrink until that cost is $\leq k$.

19. Climbing Stairs

Pattern	Rolling dynamic programming	Best container	Two integers
Time	$O(n)$	Space	$O(1)$

Core idea: Ways to reach step i equal ways to reach $i-1$ plus ways to reach $i-2$.

Why it works: The recurrence is Fibonacci-like, so only the previous two states are needed.

20. Palindrome Linked List

Pattern	Fast/slow pointers + in-place reversal	Best container	Pointers only
Time	$O(n)$	Space	$O(1)$

Core idea: Find the middle, reverse the second half, then compare corresponding nodes from the front and reversed half.

Why it works: The second-half reversal lets the list be compared from both ends without an array.

21. Lowest Common Ancestor of a Binary Tree III

Pattern	Parent pointers + depth alignment	Best container	Pointers only
Time	$O(h)$	Space	$O(1)$

Core idea: Compute both node depths, move the deeper node upward until depths match, then move both upward together.

Why it works: Once nodes are at the same depth, their first common node on the upward walk is the LCA.

22. Group Anagrams

Pattern	Hash by character-frequency signature	Best container	<code>unordered_map<</code>
Time	$O(N \cdot K)$	Space	$O(N \cdot K)$

Core idea: Count 26 letters for each word and encode those counts into a hash key. Words with identical frequency vectors go into the same bucket.

Why it works: Anagrams have exactly the same character counts regardless of order.

23. Maximum Subarray

Pattern	Kadane's algorithm	Best container	Two integers
Time	$O(n)$	Space	$O(1)$

Core idea: At each element, choose whether to start a new subarray or extend the current one. Track the best sum seen.

Why it works: A negative accumulated prefix should be discarded because it can only hurt any future subarray.

24. Reverse Words in a String

Pattern	Tokenize + reverse	Best container	vector, stringstream
Time	$O(n)$	Space	$O(n)$

Core idea: Extract whitespace-separated words, reverse their order, and join with single spaces.

Why it works: Token extraction automatically removes leading, trailing, and repeated spaces.

25. Valid Word Abbreviation

Pattern	Two pointers	Best container	No special container
Time	$O(n + m)$	Space	$O(1)$

Core idea: Walk through word and abbreviation together. Letters must match; digit groups tell how many word characters to skip.

Why it works: Parsing each numeric run as one skip amount handles multi-digit abbreviations correctly; leading zeroes are rejected.

Additional C++ Interview Solutions

Concise explanations appended to the previous study guide.

26. Find Peak Element

Pattern	Binary search	Best container	vector
Time	$O(\log n)$	Space	$O(1)$

Core idea: Compare `nums[mid]` with `nums[mid+1]`. If the slope goes down, a peak exists on the left including mid; otherwise a peak exists on the right.

Why it works: The search always keeps at least one peak inside the remaining interval.

27. Design Tic-Tac-Toe

Pattern	Row/column counters	Best container	vector plus two integers
Time	$O(1)$ per move	Space	$O(n)$

Core idea: Store a running score for each row, column, diagonal, and anti-diagonal. Add +1 for player 1 and -1 for player 2.

Why it works: A player wins exactly when the absolute value of one relevant counter reaches n .

28. Shortest Word Distance II

Pattern	Preprocess positions + two pointers	Best container	<code>unordered_map<</code>
Time	Build $O(N)$, query $O(a+b)$	Space	$O(N)$

Core idea: Store every index for each word. For a query, walk the two sorted index lists with two pointers and keep the smallest difference.

Why it works: Advancing the pointer at the smaller index is the only move that can reduce the distance.

29. Maximum Profit in Job Scheduling

Pattern	DP + binary search	Best container	<code>vector<, vector</code>
Time	$O(n \log n)$	Space	$O(n)$

Core idea: Sort jobs by start time. At each job choose between skipping it or taking its profit plus the best value from the first compatible future job.

Why it works: Binary search finds the next non-overlapping job, while DP avoids recomputing subproblems.

30. Binary Tree Maximum Path Sum

Pattern	Postorder DFS	Best container	Recursion
Time	$O(n)$	Space	$O(h)$

Core idea: For each node compute the best one-sided gain that can be extended to its parent. Also test a complete path through the node using both positive child gains.

Why it works: A parent can extend through only one child, but a complete maximum path may use both children.

31. Kth Missing Positive Number

Pattern	Binary search on missing count	Best container	vector
Time	$O(\log n)$	Space	$O(1)$

Core idea: At index i , $\text{arr}[i] - (i + 1)$ tells how many positive integers are missing before $\text{arr}[i]$. Binary-search the first index where this count reaches k .

Why it works: The missing-count function is monotonic, so binary search applies directly.

32. Basic Calculator II

Pattern	Single pass + stack	Best container	vector
Time	$O(n)$	Space	$O(n)$

Core idea: Parse each number and apply the previous operator. Push addition/subtraction terms; immediately combine multiplication/division with the stack top.

Why it works: Immediate handling of $*$ and $/$ enforces precedence before the final sum.

33. Sort Colors

Pattern	Dutch national flag	Best container	vector
Time	$O(n)$	Space	$O(1)$

Core idea: Maintain regions for 0s, unknown values, and 2s. Swap 0s left, 2s right, and advance through 1s.

Why it works: Each swap permanently places a boundary value, so one pass is enough.

34. Valid Anagram

Pattern	Frequency counting	Best container	array
Time	$O(n)$	Space	$O(1)$

Core idea: Count letters from the first string and decrement with the second. Any negative count or unequal length means the strings differ.

Why it works: Anagrams contain exactly the same frequency of every character.

35. Search Insert Position

Pattern	Lower-bound binary search	Best container	vector
Time	$O(\log n)$	Space	$O(1)$

Core idea: Search for the first index whose value is at least target. Return that index whether target exists or not.

Why it works: That boundary is exactly the correct sorted insertion position.

36. Remove All Adjacent Duplicates in String II

Pattern	Run-length stack	Best container	vector<
Time	O(n)	Space	O(n)

Core idea: Track consecutive character groups as (character,count). Remove a group immediately when its count reaches k.

Why it works: After a removal, neighboring groups can merge naturally through the stack top.

37. Minimum Remove to Make Valid Parentheses

Pattern	Stack of indices	Best container	stack, vector
Time	O(n)	Space	O(n)

Core idea: Store indices of unmatched opening parentheses. Mark unmatched closing parentheses during the scan, then mark any leftover openings and rebuild the string.

Why it works: Every kept closing parenthesis has a matching earlier opening, and every kept opening is eventually matched.

38. Number of Islands II

Pattern	Dynamic Union-Find	Best container	vector parent/rank
Time	O(k alpha(mn))	Space	O(mn)

Core idea: Activate each new land cell as its own component, then union it with active neighbors. Increase the island count for new land and decrement it for every successful merge.

Why it works: Union-Find maintains connected components incrementally without rescanning the entire grid.

39. Remove Element

Pattern	In-place write pointer	Best container	vector
Time	O(n)	Space	O(1)

Core idea: Scan the array and copy only values different from val to the next write position. Return the number of kept elements.

Why it works: Every valid element is compacted into the prefix exactly once.

40. IP to CIDR

Pattern	Bit manipulation + greedy blocks	Best container	uint32_t, vector
Time	O(number of output blocks)	Space	O(number of output blocks)

Core idea: Convert the IP to a 32-bit integer. Repeatedly choose the largest aligned power-of-two block that does not exceed the remaining address count.

Why it works: CIDR blocks must be power-of-two sized and aligned; the lowest set bit gives the maximum alignment-supported block.

41. Minimum Absolute Difference

Pattern	Sort + adjacent scan	Best container	vector
Time	$O(n \log n)$	Space	$O(1)$ extra

Core idea: Sort the array, find the smallest gap between adjacent values, then collect every adjacent pair with that gap.

Why it works: After sorting, the closest pair must be adjacent.

42. Vertical Order Traversal of a Binary Tree

Pattern	DFS + coordinate sort	Best container	vector<>
Time	$O(n \log n)$	Space	$O(n)$

Core idea: Record each node as (column,row,value), sort those triples, then group consecutive entries by column.

Why it works: Tuple ordering matches the required priority: column first, then row, then value.

43. 3Sum

Pattern	Sort + two pointers	Best container	vector
Time	$O(n^2)$	Space	$O(1)$ extra

Core idea: Sort the array, fix one value, then use two pointers to find complementary pairs summing to its negative. Skip duplicates.

Why it works: Sorting makes pointer movement monotonic and duplicate elimination straightforward.

44. First Missing Positive

Pattern	Index placement	Best container	vector
Time	$O(n)$	Space	$O(1)$

Core idea: While possible, place value x in index $x-1$ for values in $[1,n]$. Then the first index not containing $i+1$ reveals the answer.

Why it works: Only values 1 through n can affect the smallest missing positive; correct placement turns the array into a direct membership map.

45. Open the Lock

Pattern	Breadth-first search	Best container	queue, unordered_set
Time	$O(10^4)$	Space	$O(10^4)$

Core idea: Treat each lock state as a graph node. BFS from 0000, generating the eight one-wheel neighbors while skipping deadends and visited states.

Why it works: BFS explores states in increasing number of turns, so the first time target is reached is optimal.

46. Majority Element

Pattern	Boyer-Moore voting	Best container	Two integers
Time	$O(n)$	Space	$O(1)$

Core idea: Keep a candidate and balance counter. Matching values increase the counter; different values decrease it; choose a new candidate when the counter reaches zero.

Why it works: Because the majority occurs more than half the time, pairwise cancellation cannot eliminate all of its occurrences.